



Full Project Code Documentation & Explanation

Covers: [main.py](#) · [voice_emotion/](#) · [walkthrough.ipynb](#)
Generated: June 02, 2026

Module	File(s)	Role
main.py	main.py	Real-time orchestrator (webcam + mic)
Voice Features	voice_emotion/features.py	Extract MFCC / Chroma / Spectral Contrast
Voice Model	voice_emotion/model.py	Stacked LSTM classifier
Voice Train	voice_emotion/train.py	Train LSTM on RAVDESS + CREMA-D + TESS
Voice Predict	voice_emotion/predict.py	Inference wrapper for WAV files
Notebook	walkthrough.ipynb	Step-by-step teaching walkthrough

Technology Stack:

Python 3.10

TensorFlow / Keras

OpenCV

librosa

PyAudio

Table of Contents

- 1. Project Overview & Architecture**
- 2. main.py — Real-Time Orchestrator**
 - 2.1 Imports & Predictor Loading
 - 2.2 Audio Recording Thread
 - 2.3 Overlay Drawing
 - 2.4 Main Loop
 - 2.5 Final Session Report
- 3. voice_emotion/features.py — Feature Extraction**
 - 3.1 Label Parsers
 - 3.2 extract_features()
 - 3.3 Dataset Loaders
- 4. voice_emotion/model.py — LSTM Architecture**
 - 4.1 Model Design
 - 4.2 Layer-by-Layer Breakdown
- 5. voice_emotion/train.py — Training Pipeline**
 - 5.1 Data Normalisation
 - 5.2 Train/Val/Test Split
 - 5.3 Callbacks
 - 5.4 Evaluation
- 6. voice_emotion/predict.py — Inference**
 - 6.1 VoiceEmotionPredictor Class
 - 6.2 Predict Flow
- 7. walkthrough.ipynb — Jupyter Notebook Guide**
 - 7.1 Dataset Exploration
 - 7.2 Face CNN Section
 - 7.3 Voice LSTM Section
 - 7.4 Confidence Estimation
 - 7.5 End-to-End Inference
- 8. Data Flow Diagram**
- 9. Key Concepts Glossary**

1. Project Overview & Architecture

The **Interview Emotion Analyzer** is an AI-powered real-time tool that observes a job candidate during an interview through two independent channels — **facial expressions** via webcam and **vocal tone** via microphone — and fuses these signals to produce a live confidence/stress report. The final output tells the interviewer (or coach) whether the candidate appears calm, stressed, or engaged.

Layer	Component	Technology	Output
Input	Webcam frames	OpenCV VideoCapture	BGR image array
Input	Microphone audio	PyAudio stream	WAV segments (3 s)
Model A	Face CNN	TensorFlow/Keras Conv2D	Emotion + confidence
Model B	Voice LSTM	TensorFlow/Keras LSTM	Emotion + confidence
Fusion	Confidence Engine	confidence_analysis/	Score, stress, comm
Output	Live overlay	OpenCV imshow	Annotated video window
Output	Session report	Console / PDF	Summary statistics

2. main.py — Real-Time Orchestrator

Purpose: This is the entry point of the application. It glues together the face predictor, the voice predictor, the confidence engine, and the OpenCV display window into a single real-time loop. Think of it as the conductor that tells every other module when to play.

■ *Run with: `python main.py` — Requires trained models in the `models/` folder. Press `Q` inside the video window to stop and see the session report.*

2.1 Imports & Predictor Loading

The top of the file imports all necessary libraries and then **eagerly loads both ML models** before entering the main loop. If the face model file is missing it exits immediately with a helpful error, while the voice model is treated as optional — the app still runs with face-only analysis.

```
import os, sys, time, threading, queue

import cv2

import numpy as np

from utils.config import FACE_MODEL_PATH, VOICE_MODEL_PATH
from confidence_analysis.confidence import compute_confidence, overall_recommendation

# Face model is REQUIRED — exit if missing
if not os.path.exists(FACE_MODEL_PATH):
    print('[ERROR] Face model not found: ' + FACE_MODEL_PATH)
    sys.exit(1)

from face_emotion.predict import FaceEmotionPredictor
face_pred = FaceEmotionPredictor()

# Voice model is OPTIONAL — warn and continue without it
VOICE_AVAILABLE = os.path.exists(VOICE_MODEL_PATH)
if VOICE_AVAILABLE:
    from voice_emotion.predict import VoiceEmotionPredictor
    voice_pred = VoiceEmotionPredictor()
```

Why eager loading? ML models (especially those based on TensorFlow) take several seconds to load weights into memory. Loading them before the loop starts ensures there is zero lag on the first frame, giving a smooth user experience.

2.2 Audio Recording Thread

Audio capture runs on a **separate daemon thread** so that it never blocks the video loop. The thread records 3-second segments, saves each to a temporary WAV file, runs the LSTM predictor, and puts the result into a thread-safe queue.Queue.

```

def audio_recording_loop(result_queue, stop_event,
                          segment_duration=3.0, sr=22050):
    pa = pyaudio.PyAudio()
    stream = pa.open(format=pyaudio.paInt16, channels=1,
                      rate=sr, input=True, frames_per_buffer=1024)

    while not stop_event.is_set():
        frames = []
        for _ in range(int(sr / 1024 * segment_duration)):
            frames.append(stream.read(1024, exception_on_overflow=False))

        # Normalise raw bytes to float32 in [-1, 1]
        audio_data = np.frombuffer(b''.join(frames), dtype=np.int16)
        audio_data = audio_data.astype(np.float32) / 32768.0

        # Write to temp file, predict, clean up
        with tempfile.NamedTemporaryFile(suffix='.wav', delete=False) as tmp:
            sf.write(tmp.name, audio_data, sr)
            result = voice_pred.predict_file(tmp.name)
            os.unlink(tmp.name)      # delete after use

        result_queue.put(result)     # non-blocking push to main thread

```

Key design decisions:

- **daemon=True** — the thread dies automatically when the main process exits, so no cleanup code is needed.
- **stop_event (threading.Event)** — a shared flag lets the main loop signal the audio thread to stop gracefully.
- **queue.Queue** — Python's built-in thread-safe FIFO. The main loop reads from it non-blocking with `get_nowait()`.
- **Temporary files** — librosa/soundfile need a real file on disk, so we write, predict, then immediately delete.
- **Division by 32768** — PyAudio returns 16-bit signed integers (range -32768 to +32767). Dividing normalises to [-1,1] float, which librosa and the model expect.

2.3 Overlay Drawing — draw_info_panel()

This helper renders a semi-transparent HUD (Heads-Up Display) at the bottom of each video frame. It uses OpenCV's **alpha blending** trick: draw a solid black rectangle on a copy of the frame, then blend it back with the original using `addWeighted`.

```

def draw_info_panel(frame, face_res, v_res, conf_report):
    h, w = frame.shape[:2]
    overlay = frame.copy()          # work on a copy
    cv2.rectangle(overlay, (0, h-120), (w, h), (0,0,0), -1) # black bar
    cv2.addWeighted(overlay, 0.6, frame, 0.4, 0, frame)      # 60% opacity

    # Draw three lines of text

```

```

cv2.putText(frame, face_txt, (10, h-85), ..., (100,255,100), 2) # green
cv2.putText(frame, voice_txt, (10, h-55), ..., (100,200,255), 2) # cyan
cv2.putText(frame, conf_txt, (10, h-20), ..., (255,215,0), 2) # gold

```

2.4 Main Loop

The main loop reads frames from the webcam at ~30 fps, runs face detection on each frame, checks for fresh voice results, and recomputes the confidence report every 30 frames (~1 second).

```

while True:
    ret, frame = cap.read()

    # 1. Face: annotate the frame and collect result
    annotated, face_results = face_pred.annotate_frame(frame)
    if face_results:
        face_history.append(face_results[0])

    # 2. Voice: drain the queue (non-blocking)
    if not result_queue.empty():
        voice_result = result_queue.get_nowait()
        voice_history.append(voice_result)

    # 3. Confidence: recalculate every 30 frames
    if len(face_history) % 30 == 0 and face_history:
        conf_report = compute_confidence(
            voice_history or [{'emotion': 'neutral', 'confidence': 0.5}],
            [[r] for r in face_history[-60:]] # last 60 face frames
        )

    # 4. Draw HUD and show
    annotated = draw_info_panel(annotated, top_face, voice_result, conf_report)
    cv2.imshow('Interview Emotion Analyzer [q=quit]', annotated)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

```

2.5 Final Session Report

When the user presses Q, the loop exits and the code computes a post-session summary. It uses Python's **Counter** to find the most frequently predicted emotion across all frames, then calls the confidence engine one final time on the full history.

```

from collections import Counter

# Dominant face emotion over the whole session
face_emotions = [r['emotion'] for r in face_history]
dominant_face = Counter(face_emotions).most_common(1)[0][0]

# Dominant voice emotion

```

```
voice_emotions = [r['emotion'] for r in voice_history]
dominant_voice = Counter(voice_emotions).most_common(1)[0][0]

final = compute_confidence(voice_history, [[r] for r in face_history])
print(f'Confidence Score : {final["confidence_score"]}%' )
print(overall_recommendation(final['confidence_score'], dominant_face, dominant_voice))
```

3. voice_emotion/features.py — Feature Extraction

Purpose: Convert raw WAV audio files into fixed-size numerical feature matrices that the LSTM can learn from. Raw audio (millions of samples) is far too large and noisy for a neural network to process directly — feature engineering compresses it into the most emotionally relevant information.

3.1 Label Parsers

Three popular speech-emotion datasets are supported: **RAVDESS**, **CREMA-D**, and **TESS**. Each encodes the emotion differently (filename codes vs folder names), so a dedicated parser handles each.

Dataset	Encoding	Example	Parser
RAVDESS	3rd segment of filename	03-01-03-01-01-01-01.wav → happy	label_from_ravdess()
CREMA-D	3rd underscore segment	1001_DFA_ANG_XX.wav → angry	label_from_cremad()
TESS	Folder name substring	OAF_happy/ → happy	_label_from_tess_folder()

3.2 extract_features() — The Core Function

This function is the heart of the voice module. Given a path to any WAV file, it returns a 2-D NumPy array of shape **(59, 174)** containing three stacked feature types:

Feature	Shape	What it captures	Why it matters for emotion
MFCC	(40, T)	Spectral envelope / vocal timbre	Differentiates voices; tense muscles change formant frequencies
Chroma	(12, T)	Pitch class distribution	Pitch rises with excitement/anger, drops with sadness
Spectral Contrast	(7, T)	Difference between peaks and valleys in spectrum	Clarity vs muddiness vs muddiness — high contrast = energetic speech

```
def extract_features(wav_path, n_mfcc=40, max_pad_len=174):  
    y, sr = librosa.load(wav_path, sr=22050, mono=True)  
  
    mfcc      = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc)      # (40, T)  
    chroma    = librosa.feature.chroma_stft(y=y, sr=sr)              # (12, T)  
    spec_contrast = librosa.feature.spectral_contrast(y=y, sr=sr)    # (7, T)  
  
    def pad(feat):  
        if feat.shape[1] < max_pad_len:  
            # Pad with zeros on the right to reach max_pad_len  
            return np.pad(feat, ((0, 0), (0, max_pad_len - feat.shape[1])))  
        return feat[:, :max_pad_len]    # truncate if too long  
  
    feat = np.vstack([pad(mfcc), pad(chroma), pad(spec_contrast)]) # (59, 174)  
    return feat.astype(np.float32)
```

Why pad to 174 time steps? Different audio clips have different lengths, but a neural network needs fixed-size inputs. At 22050 Hz with a default hop length of 512, about 4 seconds of audio produces ~174

frames. Clips shorter than 4 s are zero-padded; longer clips are truncated.

3.3 Dataset Loaders

Three loaders — `load_ravdess()`, `load_cremad()`, `load_tess()` — follow the same pattern: walk the directory, parse the label, extract features, skip invalid files. `load_all_voice_data()` combines all three into a single (X, y) pair ready for training.

4. voice_emotion/model.py — LSTM Architecture

Purpose: Define the neural network that maps a (59, 174) feature matrix to a probability distribution over 7 emotion classes. A Long Short-Term Memory (LSTM) network is chosen because audio is inherently sequential — the pattern over time matters, not just individual frames.

4.1 Why LSTM for Audio?

- Speech unfolds over time. The emotion in a sentence depends on how the voice *changes*, not a single frozen instant.
- LSTMs have gated memory cells that can selectively remember patterns from earlier time steps — e.g., a rising pitch that started 0.5 s ago.
- A simple Dense (fully-connected) network treats all time steps independently and loses temporal structure.

4.2 Layer-by-Layer Breakdown

```
inp = Input(shape=(59, 174))      # (n_features, time_steps)
x = Permute((2, 1))(inp)          # → (174, 59) so LSTM sees time first

x = LSTM(128, return_sequences=True,
        dropout=0.3, recurrent_dropout=0.2)(x)
# 128 hidden units; output ALL time steps for next LSTM

x = LSTM(64, dropout=0.3, recurrent_dropout=0.2)(x)
# 64 units; output ONLY the last time step

x = Dense(64, activation='relu', kernel_regularizer=l2(1e-4))(x)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)

out = Dense(7, activation='softmax', name='emotion_probs')(x)
# 7 emotions: neutral, happy, sad, angry, fear, disgust, surprise
```

Layer	Output Shape	Purpose
Input	(batch, 59, 174)	Accept feature matrix
Permute	(batch, 174, 59)	Rearrange axes so LSTM processes time axis
LSTM(128)	(batch, 174, 128)	Capture temporal patterns at high capacity
LSTM(64)	(batch, 64)	Summarise sequence into a context vector
Dense(64, ReLU)	(batch, 64)	Non-linear combination of LSTM output
BatchNorm	(batch, 64)	Stabilise activations, accelerate training
Dropout(0.4)	(batch, 64)	Randomly zero 40% of neurons — prevents overfitting
Dense(7, Softmax)	(batch, 7)	Convert to probabilities summing to 1.0

5. voice_emotion/train.py — Training Pipeline

Purpose: Load all voice data, split it correctly, normalise features, train the LSTM with best-practice callbacks, evaluate on a held-out test set, and save artefacts (model file + scaler) for use at inference time.

5.1 Data Normalisation

Raw MFCC/Chroma values can span very different ranges (e.g., MFCC values range from -200 to +200 while Chroma values are 0–1). Without normalisation, large-magnitude features dominate the gradient signal. The solution is **per-feature z-score normalisation**: subtract the mean and divide by standard deviation.

```
def normalise(X_train, X_val, X_test):  
    N_tr, F, T = X_train.shape  
    scaler = StandardScaler()  
  
    # Fit ONLY on training data – never look at val/test during fit  
    X_tr = scaler.fit_transform(X_train.reshape(N_tr, -1)).reshape(N_tr, F, T)  
  
    # Apply same transform to val and test  
    X_v = scaler.transform(X_val.reshape(X_val.shape[0], -1)).reshape(X_val.shape)  
    X_te = scaler.transform(X_test.reshape(X_test.shape[0], -1)).reshape(X_test.shape)  
  
    return X_tr, X_v, X_te, scaler # scaler saved to disk for inference
```

■ **Critical:** The scaler must **ONLY** be fitted on training data. Fitting on the full dataset would be data leakage — the model would indirectly see test-set statistics during training, giving falsely optimistic accuracy numbers.

5.2 Train / Val / Test Split

A two-stage split produces three non-overlapping sets. The **stratify** parameter ensures each split preserves the original class proportions (important when classes are imbalanced).

```
# Stage 1: carve out 15% for final test set  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.15, stratify=y, random_state=42)  
  
# Stage 2: from remaining 85%, take 15% as validation (~12.75% of total)  
X_train, X_val, y_train, y_val = train_test_split(  
    X_train, y_train, test_size=0.15, stratify=y_train, random_state=42)
```

5.3 Training Callbacks

Callback	Configuration	Why it is used
ModelCheckpoint	monitor=val_accuracy, save_best_only=True	Saves the weights only when val accuracy improves — keeps the best model
EarlyStopping	patience=10, restore_best_weights=True	Stops training if val loss hasn't improved for 10 epochs, then restore best weights

ReduceLROnPlateau	factor=0.5, patience=5, min_lr=1e-6	Halves the learning rate when val loss plateaus — helps escape local minima
-------------------	-------------------------------------	---

The model is compiled with **Adam** (lr=0.001) and **categorical_crossentropy** loss. Adam is chosen because it adapts per-parameter learning rates and converges well on noisy audio data.

6. voice_emotion/predict.py — Inference Wrapper

Purpose: Provide a clean, reusable class (VoiceEmotionPredictor) that hides all the complexity of model loading, feature extraction, and normalisation behind a simple `predict_file(wav_path)` method.

6.1 VoiceEmotionPredictor Class

```
class VoiceEmotionPredictor:
    def __init__(self, model_path=VOICE_MODEL_PATH,
                  scaler_path=SCALER_PATH):
        # Check both artefacts exist before loading
        for p in (model_path, scaler_path):
            if not os.path.exists(p):
                raise FileNotFoundError(f'Not found: {p}')

        self.model = tf.keras.models.load_model(model_path)
        self.scaler = joblib.load(scaler_path) # same scaler fitted during training
```

Loading both the model *and* the scaler together is critical: if you used the scaler during training to normalise features, you must apply the **exact same transformation** at inference time. A mismatch (e.g., forgetting to normalise) will cause wildly wrong predictions even from a perfect model.

6.2 Predict Flow

```
def predict_file(self, wav_path):
    feat = extract_features(wav_path) # → (59, 174)
    if feat is None: return {'emotion': 'unknown', ...}
    return self._predict_features(feat)

def _predict_features(self, feat):
    inp = self._normalise(feat) # apply z-score scaler
    probs = self.model.predict(inp)[0] # → (7,) probability vector
    idx = int(np.argmax(probs)) # class with highest prob
    return {
        'emotion': VOICE_EMOTIONS[idx], # e.g. 'happy'
        'confidence': float(probs[idx]), # e.g. 0.82
        'probabilities': {e: p for e, p in zip(VOICE_EMOTIONS, probs)}
    }
```

The probabilities dict lets callers see not just the winner but how confident the model is across all classes — useful for the confidence engine which weighs uncertain predictions differently from decisive ones.

7. walkthrough.ipynb — Jupyter Notebook Guide

Purpose: An interactive, self-contained teaching document. Each cell can be run independently to visualise data, inspect model architectures, and test predictions — without modifying production code. It is structured as a guided tour across 5 sections.

7.1 Dataset Exploration

The first section counts images per emotion class in the FER2013 dataset and plots both a bar chart (class distribution) and sample face images from each class. This is important for spotting **class imbalance** — if 'neutral' has 5× more samples than 'disgust', the model will be biased toward neutral unless we apply class weighting or data augmentation.

```
for emotion in FACE_EMOTIONS:
    path = os.path.join(FER_TRAIN_DIR, emotion)
    n = len(os.listdir(path))
    counts[emotion] = n
    print(f'{emotion:12s}: {n:5d} images')

plt.bar(counts.keys(), counts.values(), color='steelblue')
plt.title('FER2013 Class Distribution')
```

7.2 Face CNN Section

Calls `build_face_cnn()` and prints `model.summary()`, then optionally renders a visual architecture diagram using `tf.keras.utils.plot_model()`. The cell also prints plain-English explanations of each CNN concept used.

- **Conv2D** — learns spatial patterns (edges, textures) from pixel neighborhoods
- **BatchNormalization** — normalises layer activations to stabilise and speed up training
- **MaxPooling** — downsamples feature maps, reducing computation while keeping dominant features
- **Dropout** — randomly deactivates neurons during training to prevent overfitting
- **Softmax** — converts final layer scores into a probability distribution across 7 emotion classes

7.3 Voice LSTM Section

Loads a real audio file from the RAVDESS dataset, extracts its MFCC, and visualises the result as a spectrogram heatmap using `librosa.display.specshow()`. This makes the abstract feature extraction concrete — you can literally see how the audio changes over time.

```
y, sr = librosa.load(sample_wav, sr=22050)
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=40)

librosa.display.specshow(mfcc, x_axis='time', sr=sr)
plt.colorbar()
```

```
plt.title(f'MFCC - {filename}')
```

7.4 Confidence Estimation Section

Uses **mock data** (hand-crafted fake predictions) to demonstrate the confidence engine without needing trained models. This is excellent for unit-testing the logic independently of the ML pipeline.

```
mock_voice = [  
    {'emotion': 'neutral', 'confidence': 0.78},  
    {'emotion': 'neutral', 'confidence': 0.82},  
    {'emotion': 'happy', 'confidence': 0.65},  
]  
  
mock_faces = [[{'emotion': 'happy', 'confidence': 0.80}], ...]  
  
report = compute_confidence(mock_voice, mock_faces)  
  
print(overall_recommendation(report['confidence_score'], 'happy', 'neutral'))
```

7.5 End-to-End Inference Test

The final cell checks whether trained model files exist on disk and, if so, instantiates both predictors and prints usage instructions. This acts as a **readiness check** before running main.py — it clearly shows whether training is still pending.

8. Complete Data Flow

The diagram below shows how data moves through the entire system from raw inputs to the final report:

Step	Component	Input	Output
1	OpenCV VideoCapture	Webcam (USB/built-in)	BGR frame (H×W×3)
2	FaceEmotionPredictor annotate_frame()	BGR frame	Annotated frame + list of {emotion, confidence}
3	PyAudio stream	Microphone	Raw int16 bytes
4	Normalise audio	int16 bytes	float32 array in [-1,1]
5	soundfile.write()	float32 array	Temp .wav file
6	extract_features()	WAV path	NumPy (59,174) matrix
7	StandardScaler .transform()	(59,174) raw	(59,174) z-scored
8	LSTM model.predict()	(1,59,174) tensor	(1,7) softmax probs
9	argmax + label	(7,) probs	{emotion, confidence, all_probs}
10	compute_confidence()	face & voice history	{score, stress, comm}
11	draw_info_panel()	frame + all results	HUD-annotated frame
12	cv2.imshow()	Annotated frame	Live display window
13	Session report	Full history	Console summary text

9. Key Concepts Glossary

MFCC (Mel-Frequency Cepstral Coefficients)

A compact representation of the short-term power spectrum of audio. Mimics human hearing by using a mel-scale (logarithmic frequency axis). Captures vocal timbre — the quality that makes a happy voice sound different from a sad one at the same pitch.

Chroma Features

Represent the energy distribution across the 12 pitch classes (C, C#, D, ..., B). Useful for detecting emotional speech patterns that correlate with musical pitch content.

Spectral Contrast

Measures the difference in amplitude between spectral peaks and valleys. High contrast correlates with clear, energetic speech; low contrast with mumbled or flat delivery.

LSTM (Long Short-Term Memory)

A type of recurrent neural network cell with three gates (input, forget, output) that control what information is retained over time. Solves the vanishing gradient problem that prevents standard RNNs from learning long-range dependencies.

CNN (Convolutional Neural Network)

Uses small sliding filters (kernels) to detect local patterns in images. Multiple layers of convolution build a hierarchy of features from edges → textures → parts → objects → emotions.

Softmax

A mathematical function that converts a vector of raw scores (logits) into probabilities that sum to 1.0. Allows the model to express both its prediction and its confidence.

Dropout

A regularisation technique that randomly sets a fraction of neuron activations to zero during each training step. Forces the network to learn redundant representations, making it more robust and preventing overfitting.

BatchNormalization

Normalises the inputs of each layer across the mini-batch, keeping activations in a stable range. This allows higher learning rates and significantly speeds up convergence.

Data Leakage

The mistake of allowing test or validation data to influence model training (e.g., fitting a scaler on all data including test). Causes falsely optimistic performance metrics.

Threading / Queue

Python's threading module allows concurrent execution. `queue.Queue` is a FIFO container safe for use across threads — the audio thread pushes results; the video loop pops them.

z-Score Normalisation

For each feature, subtract its mean and divide by its standard deviation (computed on training data only). Results in zero mean and unit variance, helping gradient-based optimisers converge faster and more reliably.

FER2013

Facial Expression Recognition 2013 — a public dataset of 48×48 grayscale face images labelled with 7 emotions: angry, disgust, fear, happy, neutral, sad, surprise.

RAVDESS

Ryerson Audio-Visual Database of Emotional Speech and Song — 24 actors, 8 emotions, high quality studio recordings, widely used for speech emotion recognition benchmarks.